

Основы работы со ссылками и границами

данном уроке мы с вами продолжим изучать различные CSS свойства.

Блок 1. Свойство text-decoration

Свойство **text-decoration** позволяет задавать некоторые эффекты для текста: подчеркивание, ~~нечеркивание~~, линию сверху, а также отменять такие эффекты, если какой-либо тег имеет их по умолчанию.

Чаще всего это свойство используются для отмены подчеркивания ссылок (они по умолчанию подчеркнуты).

Давайте посмотрим, какие значения может принимать свойство **text-decoration**.

Значение underline

Значение **underline** добавляет подчеркивание тексту:

```
p {  
    text-decoration: underline;  
}
```

<p>

Lorem ipsum dolor sit amet.

</p>

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet.

Значение overline

Значение **overline** добавляет линию над текстом:

```
p {  
    text-decoration: overline;  
}
```

<p>

Lorem ipsum dolor sit amet.

</p>

Так код будет выглядеть в браузере:

Lorem ipsum dolor sit amet.

Значение **line-through**

Значение **line-through** добавляет линию, перечеркивающую текст:

```
p {  
    text-decoration: line-through;  
}  
  
<p>  
    Lorem ipsum dolor sit amet.  
</p>
```

Так код будет выглядеть в браузере:

~~Lorem ipsum dolor sit amet.~~

Значение **none**

Значение **none** отменяет все эффекты, обычно используется для отмены подчеркивания ссылок.

В следующем примере ссылка по умолчанию будет подчеркнута, а вторая ссылка с **id="link"** будет без подчеркивания, так как мы ей зададим **text-decoration** в значении **none**:

```
#link {  
    text-decoration: none;  
}  
  
<a href="#">Ссылка по умолчанию</a>  
<a id="link" href="#">Ссылка без подчеркивания</a>
```

Так код будет выглядеть в браузере:

Ссылка по умолчанию Ссылка без подчеркивания

Блок 2 Состояния ссылок

Я думаю, что вы, посещая различные сайты в интернете, обращали внимание на то, что ссылки обычно реагируют на наведение мышкой на них. Такого эффекта можно добиться, задавая поведение ссылок в различных состояниях.

К примеру, вот так - **a:hover** - мы поймем состояние, когда на ссылку навели курсор мышки. В этот момент мы можем, к примеру, поменять цвет ссылки или убрать/добавить ей подчеркивание.

Конструкция **:hover** называется *псевдоклассом*. Псевдоклассы позволяют отлавливать разные состояния элементов.

Кроме **:hover** если еще псевдоклассы **:link**, которые отлавливают не посещенную ссылку, и **:visited**, которые отлавливают посещенную ссылку.

На некоторых сайтах с их помощью показывают пользователям, где они были, а где - нет.

Есть еще и псевдокласс **:active**, который отлавливает следующее состояние: *на элемент нажали мышкой, но еще не отпустили*.

В следующем примере для ссылки в состоянии **:hover** убирается подчеркивание, в состоянии **:link** будет красный цвет, в состоянии **:visited** - зеленый, в **:active** - голубой:

```
a:link {
    color: red;
}
a:visited {
    color: green;
}
a:hover {
    text-decoration: none;
}
a:active {
    color: blue;
}
<a href="#">Ссылка</a>
```

Так код будет выглядеть в браузере:

[Ссылка](#)

В начале ссылка будет красного цвета, после нажатия на нее - зеленого, если нажать на нее мышкой и не отпускать - голубого, а если навести мышкой - станет неподчеркнутой.

Псевдоклассы наследуют друг от друга. К примеру, если я уберу подчеркивание для состояния **:link**, то оно уберется для всех состояний.

Из-за наследования для корректной работы данные псевдоклассы следует размещать именно в таком порядке, как в примере: **:link**, **:visited**, **:hover**, **:active** (ненужные можно не писать). Этот порядок подчиняется следующему мнемоническому правилу: **LoVe HAte**.

Часто состояния **:link** и **:visited** объединяют вместе через запятую:

```
a:link, a:visited {
    color: red;
}
a:hover {
    text-decoration: none;
}
a:active {
    color: blue;
}
```

В таком случае можно их вообще и не указывать:

```
a {
    color: red;
}
a:hover {
    text-decoration: none;
}
a:active {
    color: blue;
}
```

Блок 3. Сложные селекторы с учетом состояний ссылок

Наверняка на сайте у вас будут ссылки разных видов и, чтобы отличить их друг от друга, вы будете давать им разные классы или ложить в блоки с определенным id.

Давайте научимся обращаться к таким ссылкам.

Пусть у нас есть ссылки с классом **.test** и без него. Выберем только ссылки с этим классом:

```
<a href="#">Ссылка без класса</a>
<a class="test" href="#">Ссылка с классом test</a>
<a class="test" href="#">Ссылка с классом test</a>
<a class="test" href="#">Ссылка с классом test</a>
a:link.test, a:visited.test {
```

```
        color: red;
    }
    a:hover.test {
        color: blue;
    }
```

Пусть у нас есть ссылки внутри блока с id **test**. Выберем только ссылки только из этого блока:

```
<a href="#">Ссылка вне блока</a>
<div id="test">
    <a href="#">Ссылка внутри блока</a>
    <a href="#">Ссылка внутри блока</a>
    <a href="#">Ссылка внутри блока</a>
</div>
#test a:link, #test a:visited {
    color: red;
}
#test a:hover {
    color: blue;
}
```

Блок4. Работа с границами на CSS

Сейчас мы с вами научимся добавлять **границу** элементам. Это делается при помощи трех свойств: свойство **border-width** задает толщину границы, **border-color** - цвет, а **border-style** задает тип границы.

Первые два свойства работают очевидным образом: **border-color** принимает цвета в том же формате, что и свойство **color**, а толщина границы может задаваться в любых единицах измерения (кроме процентов), чаще всего в пикселях.

А вот свойство **border-style** может принимать одно из нескольких значений: **solid** (сплошная линия), **dotted** (граница в виде точек), **dashed** (граница в виде черточек), **ridge** (выпуклая граница), **double** (двойная граница), **groove** (вогнутая граница), **inset** (рамка), **outset** (рамка) или **none** (отменяет границу).

Сделаем, к примеру, границу толщиной 3 пикселя, в виде точек, красного цвета:

```
div {  
    border-width: 3px; /* толщина 3px */  
    border-style: dotted; /* в виде точек */  
    border-color: red; /* красный цвет */  
    width: 300px;  
    height: 100px;  
}
```

Так код будет выглядеть в браузере:



Давайте теперь посмотрим на примерах, как выглядят различные типы границы.

Давайте теперь посмотрим на примерах, как выглядят различные типы границы.

Значение **solid**

Значение **solid** - сплошная линия:

```
div {  
    border-width: 1px;  
    border-style: solid;  
    border-color: black;  
    width: 300px;  
    height: 100px;  
}
```

Так код будет выглядеть в браузере:



Значение **dotted**

Значение **dotted** - линия в виде точек:

```
div {  
    border-width: 1px;  
    border-style: dotted;  
    border-color: black;  
    width: 300px;  
    height: 100px;  
}
```

Так код будет выглядеть в браузере:



Значение **dashed**

Значение **dashed** - линия в виде тире:

```
div {  
    border-width: 1px;  
    border-style: dashed;  
    border-color: black;  
    width: 300px;
```

```
height: 100px;  
}
```

Так код будет выглядеть в браузере:



Значение ridge

Значение **ridge** - выпуклая линия:

```
div {  
    border-width: 3px;  
    border-style: ridge;  
    border-color: black;  
    width: 300px;  
    height: 100px;  
}
```

Так код будет выглядеть в браузере:

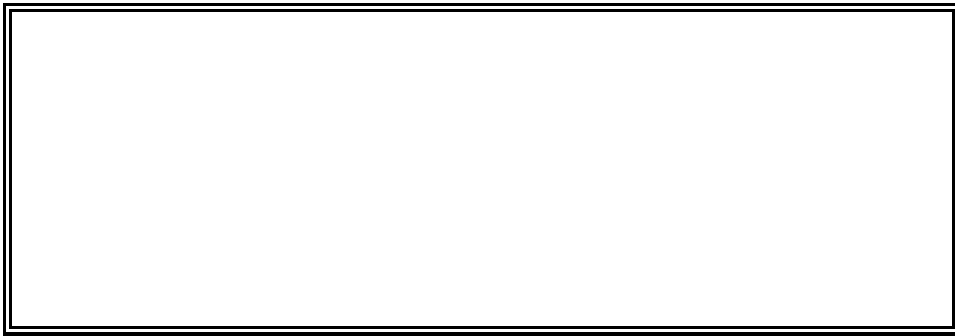


Значение double

Значение **double** - двойная линия:


```
div {  
    border-width: 3px;  
    border-style: double;  
    border-color: black;  
    width: 300px;  
    height: 100px;  
}
```

Так код будет выглядеть в браузере:



И тд есть еще свойства groove, inset, outset

Блок 5. Свойство-сокращение для границ

Так же, как и для шрифтов, для границ тоже существует **свойство-сокращение border**, которое мы можем использовать вместо того, чтобы писать 3 разных свойства для **толщины**, **цвета** и **типа** границы. В свойстве **border** мы можем просто перечислить эти значения, их порядок при этом не важен. Смотрите пример:

```
<div></div>  
div {  
    width: 300px;  
    height: 100px;  
    border: 1px solid red;  
}
```

Так код будет выглядеть в браузере:



Блок 6 Граница для отдельных сторон

Существуют также свойства-сокращения для отдельных сторон: **border-left** (левая граница), **border-right** (правая граница), **border-top** (верхняя граница), **border-bottom** (нижняя граница).

Давайте сделаем блоку **только левую границу** с помощью свойства **border-left**:

```
<div></div>
```

```
div {  
    width: 300px;  
    height: 100px;  
    border-left: 1px solid red;  
}
```

Так код будет выглядеть в браузере:



А теперь одновременно сделаем и **левую**, и **правую** границы:

```
<div></div>
```

```
div {  
    width: 300px;  
    height: 100px;  
    border-left: 1px solid red;  
    border-right: 1px solid red;  
}
```

Так код будет выглядеть в браузере:

Блок 7. Скругленные уголки

Сейчас мы с вами научимся **скруглять уголки** у границ. Это избавит наши сайты от некоторой угловатости и придаст им плавность линий.

Уголки границ (и фона, который мы разберем ниже) скругляются свойством **border-radius**, которое принимает значение в **пикселях** или **процентах** (или других единицах CSS).

Давайте скруглим уголки блоку, задав ему скругление в **10px**:

```
<div></div>

div {
    width: 300px;
    height: 100px;
    border: 1px solid red;
    border-radius: 10px;
}
```

Так код будет выглядеть в браузере:



Что означает то, что мы указали скругление в **10px**? Это **радиус круга**, который можно вписать в это скругление. Если у вас нелады с математикой и вам не понятно последнее предложение - забудьте о нем и просто подбирайте скругление на глаз. При некотором опыте это сделать не проблема (измерительного инструмента для измерения скруглений не существует, по крайней мере я о таком не слышал).

Имейте ввиду, что **border-radius** не входит в свойство-сокращение **border**, это разные свойства, хоть и имеют похожие названия.

Блок 8. Разные скругления для разных углов

Сейчас мы с вами научимся делать разные скругления для разных углов. Как это сделать: свойство **border-radius** может не только одно значение, но и **два, три** или **четыре**. Каждое значение будет задавать скругление для своего угла. Давайте посмотрим более подробно.

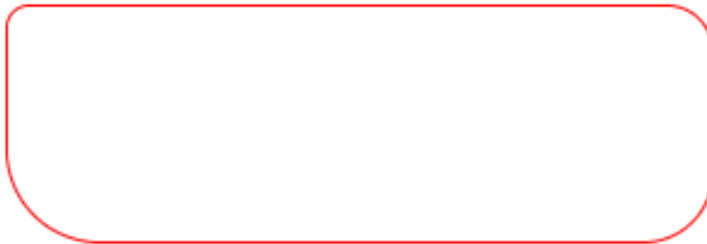
Четыре значения

Свойство **border-radius** может принимать 4 значения. Как в этом случае будут скругляться уголки - смотрите на следующем примере: **border-radius: 10px 20px 30px 40px** - **10px** - верхний левый угол, **20px** - верхний правый угол, **30px** - нижний правый угол, **40px** - нижний левый угол.

Смотрите, что у нас получится:

```
<div></div>
div {
  width: 300px;
  height: 100px;
  border: 1px solid red;
  border-radius: 10px 20px 30px 40px;
}
```

Так код будет выглядеть в браузере:



Практическая работа 4

Работа с ссылками и границами.

На text-decoration

Для решения задач данного блока вам понадобятся следующие CSS свойства: [text-decoration](#).

Повторите страницу по данному по образцу:



На границы

Для решения задач данного блока вам понадобятся следующие CSS свойства: [border-style](#), [border-width](#), [border-color](#), [border](#), [border-radius](#).

Повторите страницу по данному по образцу:



Повторите страницу по данному по образцу:



Повторите страницу по данному по образцу:

